Algorithm 計算機演算法
Class 2: Divide and Conquer

逢甲大學 人工智慧研究中心
資 訊 工 程 學 系

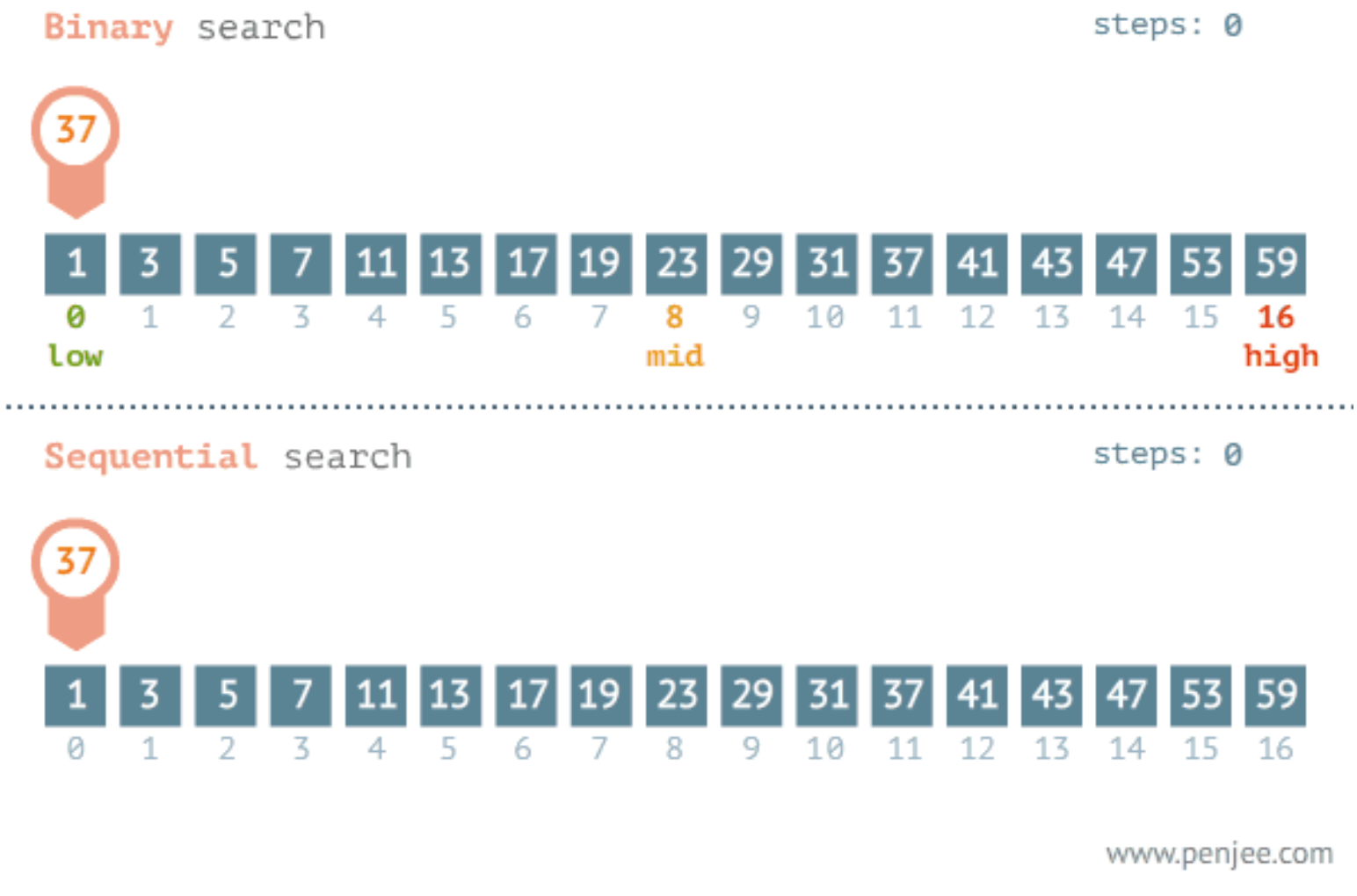
許懷中

Divide-and-Conquer（各個擊破）

Divide-and-Conquer (cont.)

- 將一個問題分割成兩個以上較小的問題
- 較小的問題通常是原問題的實例 (instance)
  - E.g. 同樣的問題，只是規模較小
- 解決**容易解決**的小問題，合併成大問題的解
- Top-down 解題
  - 以遞迴方式思考

Binary Search (二元搜尋)



Binary Search (cont.)

問題：判斷 x 是否在大小為 n 的已排序陣列 S 中。

輸入：正整數 n，以非遞減的順序排序的陣列 S（索引值由 1 到 n），以及 key$x$。

輸出：*location*x 在 S 中的位置（如果 x 不在 S 中，將傳回 0）。

```
index location (index low, index high)
{
  index mid;

  if (low > high)
    return 0;
  else {
    mid = ⌊(low + high)/2⌋;
    if (x == S[mid])
      return mid
    else if (x < S[mid])
      return location (low, mid - 1);
    else
      return location (mid + 1, high);
  }
}
```

Recursion vs. Iteration

- 上述遞迴演算法是 **tail-recursion**
- tail-recursion 的遞迴演算法可以輕易地改寫為更有效率的 iterative 演算法
  - Iterative 演算法不需要維護堆疊、也沒有 context switch 造成的 redundancy

何為 Tail-Recursion

- 比較下面兩種演算法

```
int recsum(int x) {
  if(x == 1)
    return x;
  else
    return x + recsum(x - 1);
}
```

```
int tailrecsum(int x, int runningtotal = 0) {
  if(x == 0)
    return runningtotal;
  else
    return tailrecsum(x-1, runningtotal +x);
}
```

*http://stackoverflow.com/questions/33923/what-is-tail-recursion

何為 Tail-Recursion (cont.)

- 比較上述兩種演算法的計算過程

recsum(5)
= 5 + recsum(4)
= 5 + (4 + recsum(3))
= 5 + (4 + (3 + recsum(2)))
= 5 + (4 + (3 + (2 + recsum(1))))
= 5 + (4 + (3 + (2 + 1)))
= 15

tailrecsum(5, 0)
= tailrecsum(4, 5)
= tailrecsum(3, 9)
= tailrecsum(2, 12)
= tailrecsum(1, 14)
= tailrecsum(0, 15)
= 15

Binary Search – Worst Case Time Complexity

基本運算： x 與 $S[mid]$ 的比較。

輸入大小： n，陣列中的項目數量。

如同在 1.2 節中討論的，一個最壞的情況是 x 較所有陣列中的項目為大。若 n 是 2 的乘冪且 x 比所有陣列中的項目都大，每次遞迴呼叫可以減少的個數正好一半。例如，若 n=16，則 $mid=\lfloor(1+16)/2\rfloor=8$。因為 x 大於所有陣列中的項目，前 8 大的項目成為第一次的遞迴呼叫的輸入。同樣地，前四大的項目是第二次遞迴呼叫的輸入。我們得到下列的遞迴式：

$$W(n) = \underbrace{W\left(\frac{n}{2}\right)}_{\text{在遞迴呼叫中的比較}} + \underbrace{1}_{\text{在最上層的比較}}$$

如果 n=1 且 x 大於一個陣列項目，會造成 x 與該項目比較之後，將跟著一個 $low>hight$ 的遞迴呼叫。此時，終止條件為真，意味著比較到此為止。因此，$W(1)$ 等於 1。我們得到下列的遞迴式

$$W(n) = W\left(\frac{n}{2}\right) + 1 \quad \text{對於 } n>1\text{，} n \text{ 為 } 2 \text{ 的乘冪}$$
$$W(1) = 1$$

我們在附錄 B 中的範例 B.1 將會解出這個遞迴式。其解答為

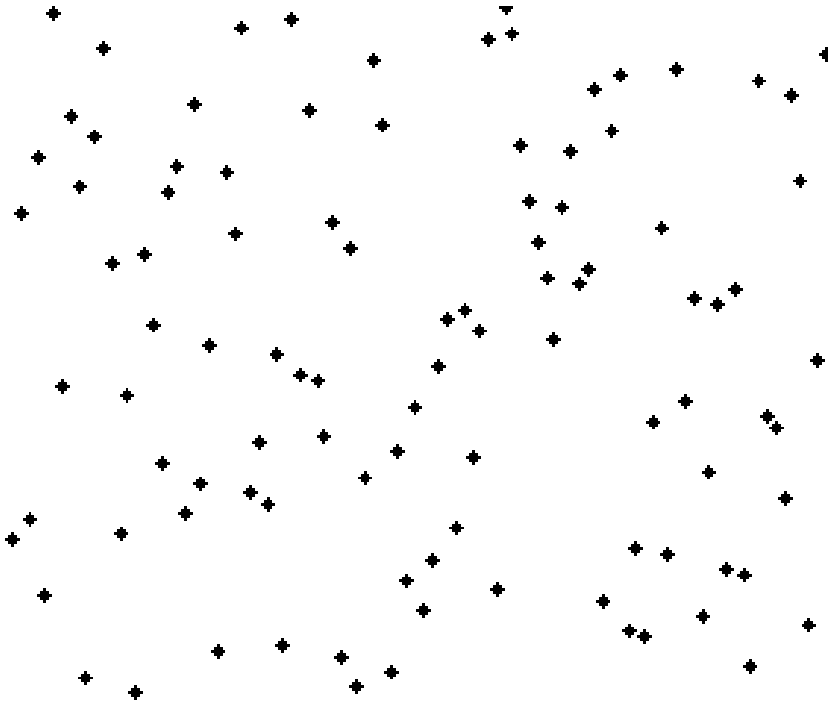
$$W(n) = \lg n + 1$$

如果 n 不限為 2 的乘冪，我們可得到

$$W(n) = \lfloor \lg n \rfloor + 1 \in \Theta(\lg n)$$

Mergesort (合併排序)

- 合併兩個已排序的陣列
  - 假如陣列夠小，直接排序
  - 否則分割 → 遞迴呼叫
  - 將已排序的子陣列合併



假設一個陣列包含了下面的數字，依序為：

$$27 \quad 10 \quad 12 \quad 20 \quad 25 \quad 13 \quad 15 \quad 22$$

1. 分割 (**Divide**) 這個陣列：

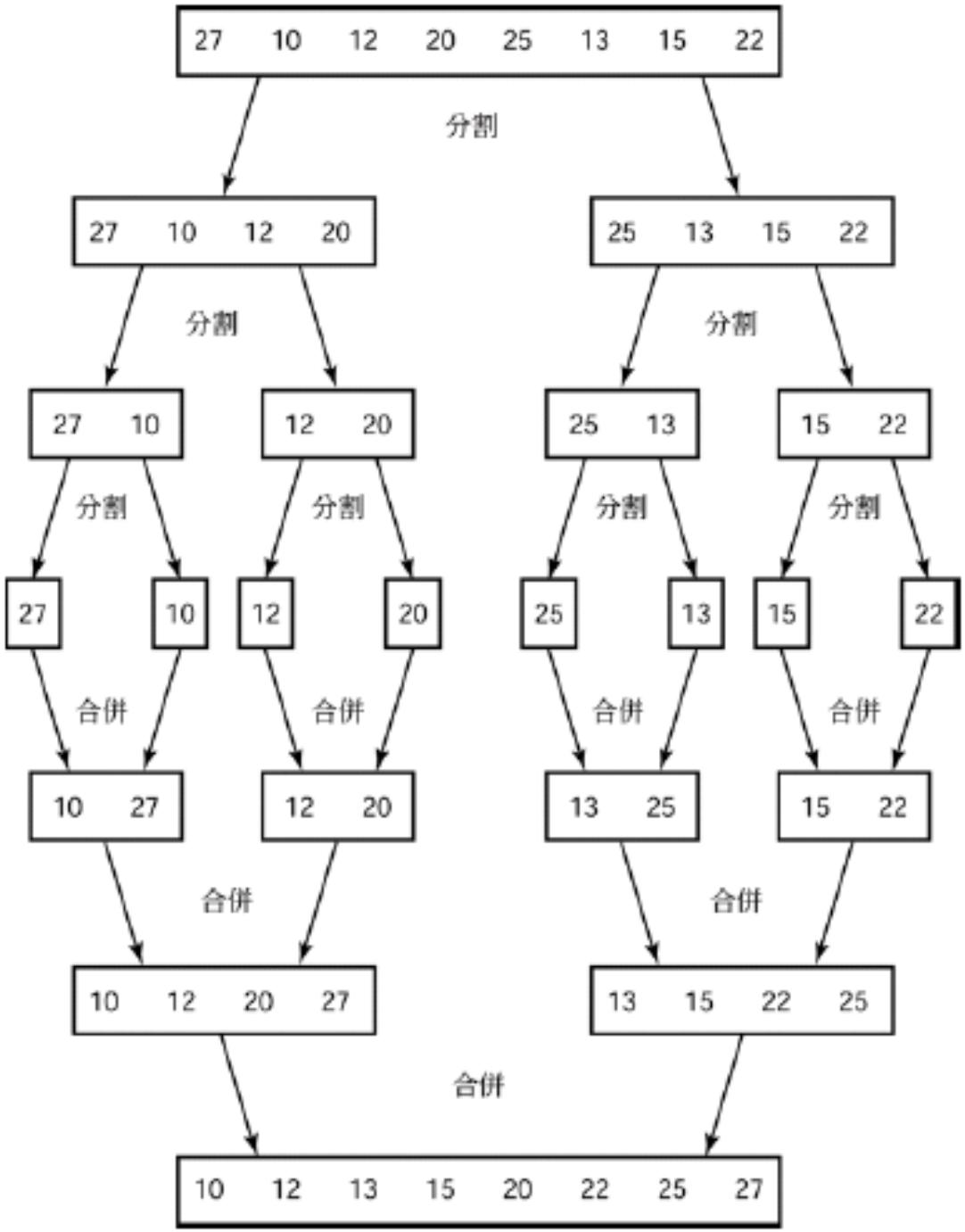
$$27 \quad 10 \quad 12 \quad 20 \quad \text{及} \quad 25 \quad 13 \quad 15 \quad 22$$

2. 對每個子陣列進行排序：

$$10 \quad 12 \quad 20 \quad 27 \quad \text{及} \quad 13 \quad 15 \quad 22 \quad 25$$

3. 合併子陣列：

$$12 \quad 13 \quad 15 \quad 20 \quad 22 \quad 25 \quad 27$$


27 10 12 20 25 13 15 22
分割
27 10 12 20
25 13 15 22
分割
分割
27 10
12 20
25 13
15 22
分割
分割
分割
分割
27
10
12
20
25
13
15
22
合併
合併
合併
合併
10 27
12 20
13 25
15 22
合併
合併
10 12 20 27
13 15 22 25
合併
10 12 13 15 20 22 25 27

Mergesort Algorithm

問題：將 n 個 key 排序成為非遞減序列。

輸入：正整數 n，鍵值陣列 S（索引值由 1 到 n）。

輸出：陣列 S，其包含的 key 已經以非遞減的順序排序過。

```
void mergesort (int n, keytype S[])
{
  if (n>1) {
      const int h = ⌊n/2⌋, m = n - h;
      keytype U[1..h], V[1..m];
      將 S[1] 至 S[h] 複製到 U[1] 至 U[h] 間；
      將 S[h+1] 至 S[h] 複製到 V[1] 至 V[m] 間；
      mergesort (h, U);
      mergesort (m, V);
      merge (h, m, U, V, S);
  }
}
```

Merge Algorithm

問題：將兩個已排序的陣列合併成一個已排序的陣列。

輸入：正整數 h 與 m，已排序的陣列 U(索引值由 1 到 h)，已排序的陣列 V(索引值由 1 到 m)。

輸入：單一已排序的陣列 S(索引值由 1 到 $h+m$)，其包含了 U 與 V 的所有 key。

```
void merge (int h, int m, const keytype U[],
                          const keytype V[],
                                keytype S[])
```

```
{
    index i, j, k;

    i = 1; j = 1; k= 1;
    while (i <= h && j <= m){
        if (U[i] < V[j]) {
            S[k] = U[i];
            i++;
        }
```

```
        else {
            s[k] = v[j];
            j++;
        }
        k++;
        }
    if (i>h)
        將 v[j] 至 v[m] 複製到 s[k] 至 s[h+m] 間;
    else
        將 u[i] 至 u[h] 複製到 s[k] 至 s[h+m] 間;
}
```

Merge Algorithm
Worst Case Time Complexity

基本運算：比較 $U[i]$ 與 $V[j]$。

輸入大小：h 與 m，分別為兩輸入陣列的項目個數。

最壞的狀況發生在當跳出迴圈時，因為 i 已經到達離開點 $h+1$，而另一個索引 j 只到達 m，還差 1 才到達離開點。舉例來說，若在 S 中先放 $m-1$ 個 V 中的項目，再放 U 中的所有 h 個項目，由於 i 等於 $h+1$，此時將會跳出迴圈。因此，

$$W(h,m) = h + m - 1$$

Mergesort
Worst Case Time Complexity

基本運算：發生在 $merge$中的比較。

輸入大小：n，陣列 S中的項目個數。

所有比較的數目為以 U為輸入對 $mergesort$的遞迴呼叫、以 V為輸入對 $mergesort$的遞迴呼叫以及最上層對 $merge$的呼叫，三者中所發生的比較次數總和。因此，

$$W(n) = \underbrace{W(h)}_{\text{將 } U \text{ 排序所花的時間}} + \underbrace{W(m)}_{\text{將 } V \text{ 排序所花的時間}} + \underbrace{h+m-1}_{\text{合併所花的時間}}$$

- n為2的乘冪的情況

$$h = \lfloor n/2 \rfloor = \frac{n}{2}$$

$$m = n - h = n - \frac{n}{2} = \frac{n}{2}$$

$$h + m = \frac{n}{2} + \frac{n}{2} = n$$

- W(n)的式子成為

$$\begin{aligned} W(n) &= W\left(\frac{n}{2}\right) + W\left(\frac{n}{2}\right) + n - 1 \\ &= 2W\left(\frac{n}{2}\right) + n - 1 \end{aligned}$$

當輸入的大小為 1 時，即達到終止條件，且無法再進行合併。因此，$W(1)$ 為 0。我們得到下列的遞迴式

$$
\begin{aligned}
&W(n) = 2W\left(\frac{n}{2}\right) + n - 1 \quad \text{當 } n>1 \text{ 且 } n \text{ 為 2 的乘冪} \\
&W(1) = 0
\end{aligned}
$$

這個遞迴式將在附錄 B 的範例 B.19 解出。其解答為

$$W(n) = n \lg n - (n-1) \in \Theta(n \lg n)$$

Space Complexity 討論

- 想一想 Mergesort 演算法要用到多少記憶體空間？可以更好嘛？
- In-place sort
  - 不使用額外空間的排序演算法
  - Mergesort 演算法有可能作到 In-place Sort 嘛？

In-place Mergesort

問題：將 n 個 key 排序成為非遞減序列。

輸入：正整數 n，key 陣列 S(索引值由 1 到 n)。

輸出：key 陣列 S，其包含所有的 key 已經以非遞減的順序排序過。

```
void mergesort2 (index low, index high)
{
  index mid;

  if (low < high) {
     mid = ⌊(low + high)/2⌋;
     mergesort2 (low, mid);
     mergesort2 (mid + 1, high);
     merge2 (low, mid, high);
  }
}
```

Merge for In-place Mergesort

問題：合併兩個在合併排序 2 中產生，S 的已排序子陣列。

輸入：索引值 low、mid 與 $high$，S 的子陣列（索引值由 low 到 $high$）。其中在 low 到 mid 與 $mid+1$ 到 $high$ 這兩個區間的 key 都已經以非遞減的順序排好。

輸出：S 的子陣列（索引值由 low 到 $high$），其中在 low 到 $high$ 區間的 key 已經以非遞減的順序排好。

```
void merge2 (index low, index mid, index high)
{
  index i, j, k;
  keytype U[low..high];            // 合併所需用到的區域陣列變數
  i = low; j = mid + 1; k = low;
  while (i ≤ mid && j ≤ high){
     if (S[i]<S[j]) {
        U[k] =S[i];
        i++;
     }
     else {
        U[k] =S[j];
        j++;
     }
        k++;
  }
  if (i > mid)
     將 S[j] 至 S[high] 複製到 U[k] 至 U[high] 間；
  else
     將 S[i] 至 S[mid] 複製到 U[k] 至 U[high] 間；
將 U[low] 至 U[high] 複製到 S[low] 至 S[high] 間；
}
```

Summary

- 分割 (Divide)
  - 大問題變小問題
- 解決 (Conquer)
  - 假如問題夠小，直接解掉
  - 否則繼續分割 (遞迴)
- 合併 (Merge)
  - 將小問題的解合併
  - 縮減時沒有分裂，則無須合併，e.g. 二元搜尋

Quicksort (快速排序法)

- 挑選樞紐 (常為陣列的第一個元素)
- 比樞紐小的放到樞紐前，反之亦然
- Divide-n-Conquer

假設某個陣列包含了下列的數值：

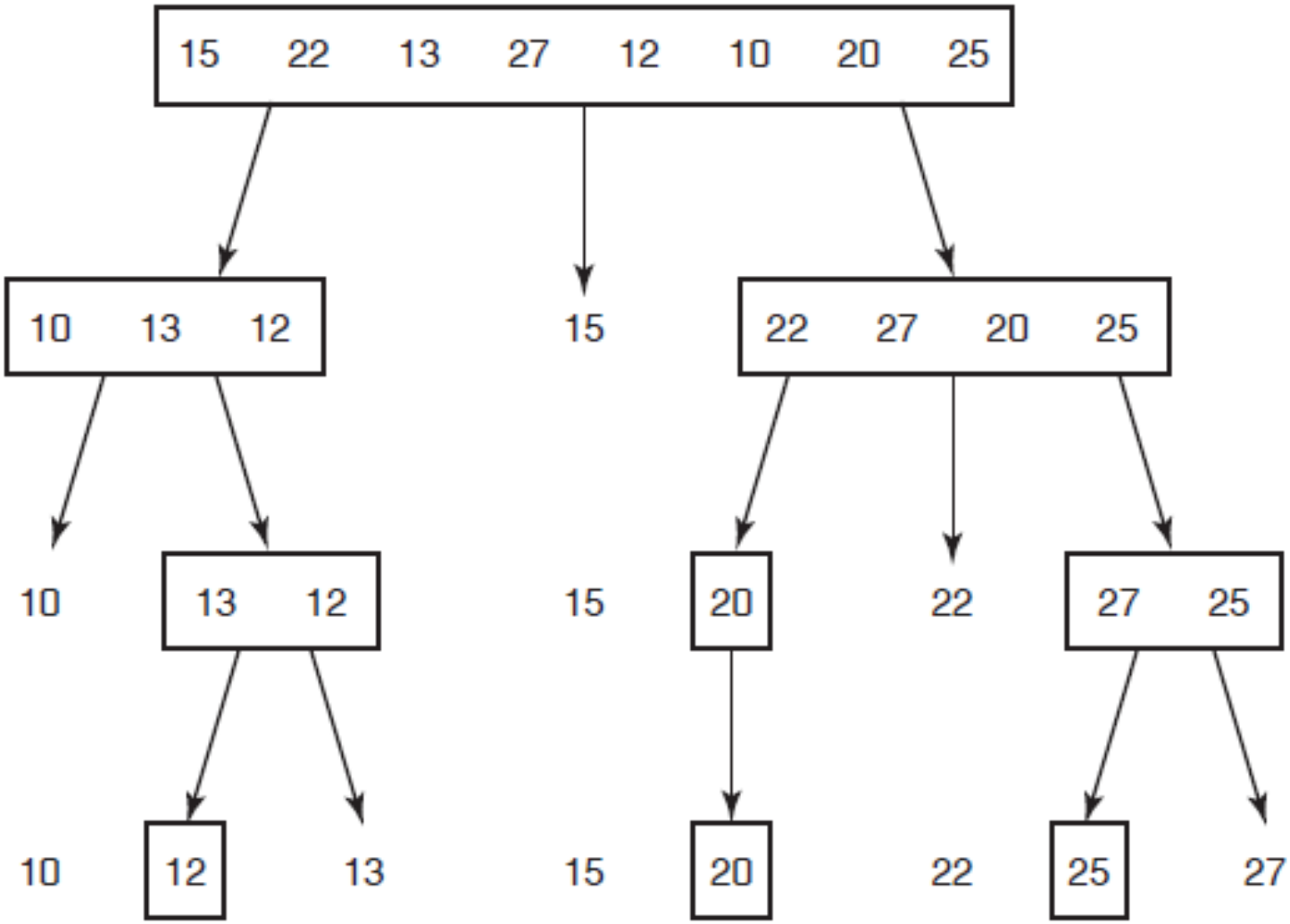
$$\begin{array}{c} \text{樞紐項} \\ \downarrow \\ 15 \quad 22 \quad 13 \quad 27 \quad 12 \quad 10 \quad 20 \quad 25 \end{array}$$

1. 分割該陣列，將小於樞紐的項目移至它的左邊而大於樞紐的項目則移至它的右邊

$$\begin{array}{c} \text{樞紐項} \\ \downarrow \\ \underbrace{10 \quad 13 \quad 12}_{\text{全部}} \quad 15 \quad \underbrace{22 \quad 27 \quad 20 \quad 25}_{\text{的項}} \end{array}$$

2. 將子陣列排序：

$$\begin{array}{c} \text{樞紐項} \\ \downarrow \\ \underbrace{10 \quad 12 \quad 13}_{\text{已排序}} \quad 15 \quad \underbrace{20 \quad 22 \quad 25 \quad 27}_{\text{已排序}} \end{array}$$


15 22 13 27 12 10 20 25
10 13 12
15
22 27 20 25
10
13 12
15
20
22
27 25
10
12
13
15
20
22
25
27

Quicksort

問題：將 n 個 key 排序成為非遞減序列。

輸入：正整數 n，key 陣列 S（索引值由 1 到 n）。

輸出：key 陣列 S，其包含所有的 key 已經以非遞減的順序排序過。

```
void quicksort (index low, index high)
{
  index pivotpoint;

  if (high > low){
     partition (low, high, pivotpoint);
     quicksort (low, pivotpoint - 1);
     quicksort (pivotpoint + 1, high);
  }
}
```

Partition

問題：將 n 個 key 排序成為非遞減序列。

輸入：正整數 n，key 陣列 S(索引值由 1 到 n)。

輸出：key 陣列 S，其包含所有的 key 已經以非遞減的順序排序過。

```
void partition (index low, index high,
                index& pivotpoint)
{
  index i, j;
  keytype pivotitem;

  pivotitem = S[low];          // 選擇第一個項目做為 pivotitem( 樞紐項 )
  j = low;
  for (i = low + 1; i <= high; i++)
     if (S[i]< pivotitem){
        j++;
        交換 S[i] 與 S[j] 的內容；
     }
  pivotpoint = j;
  交換 S[low] 與 S[pivotpoint] 的內容；     // 將 pivotitem 放在 pivotpoint
}
```

Partition: Example

i	j	S[1]	S[2]	S[3]	S[4]	S[5]	S[6]	S[7]	S[8]
-	-	15	22	13	27	12	10	20	25
2	1	**15**	**22**	13	27	12	10	20	25
3	2	**15**	22	**13**	27	12	10	20	25
4	2	**15**	$\boxed{13}$	$\boxed{22}$	**27**	12	10	20	25
5	3	**15**	13	22	27	**12**	10	20	25
6	4	**15**	13	$\boxed{12}$	27	$\boxed{22}$	**10**	20	25
7	4	**15**	13	12	$\boxed{10}$	22	$\boxed{27}$	**20**	25
8	4	**15**	13	12	10	22	27	20	**25**
-	4	$\boxed{10}$	13	12	$\boxed{15}$	22	27	20	25

Another In-Place Partition

a

6	2	8	5	11	10	4	1	9	7	3

a

6	2	3	5	11	10	4	1	9	7	8

a

6	2	3	5	1	10	4	11	9	7	8

a

6	2	3	5	1	4	10	11	9	7	8

bigElement is not to left of smallElement, terminate process.
Swap pivot and smallElement.

a

4	2	3	5	1	6	10	11	9	7	8

Partition
Every Case Time Complexity

基本運算：發生在 $merge$中的比較。

輸入大小：$n = high - low + 1$，也就是在子陣列中的項目數。

因為除第一個項目外，每個項目都會被比較，

$$T(n) = n - 1$$

因此我們使用 n 來代表子陣列的大小而非陣列 S 的大小。只有當 $partition$ 在最上層被呼叫時，它才代表 S 的大小。

Quick Sort
Worst Case Time Complexity

基本運算：在 $partition$ 副程式中，$S[i]$ 與樞紐項目的比較。

輸入大小：n，陣列 S 中的項目數。

- 最差情況：該陣列已經排成非遞減的順序

陣列不斷地被切成兩個子陣列。左邊的陣列是空的，而右邊的陣列則比原來少一個項目。

$$T(n) = \underbrace{T(0)}_{\text{排列左邊子陣列的時間}} + \underbrace{T(n-1)}_{\text{排列右邊子陣列的時間}} + \underbrace{n-1}_{\text{分割所需的時間}}$$

- 因為$T(0)=0$，故可以得到下列的遞迴式

$$T(n) = T(n-1) + n - 1 \qquad \text{for } n > 0$$
$$T(0) = 0$$

- 以歸納法推論 $T(n) = \dfrac{n(n-1)}{2}$

Quicksort
Average Case Time Complexity

Pivot 可以在任意位置

$$A(n) = \sum_{p=1}^{n} \frac{1}{n}[A(p-1) + A(n-p)] + n - 1$$

$$\sum_{p=1}^{n} [A(p-1) + A(n-p)] = 2\sum_{p=1}^{n} A(p-1)$$

$$A(n) = \frac{2}{n}\sum_{p=1}^{n} A(p-1) + n - 1$$

$$nA(n) - (n-1)A(n-1) = 2A(n-1) + 2(n-1)$$

Quicksort
Average Case Time Complexity

$$\frac{A(n)}{n+1} = \frac{A(n-1)}{n} + \frac{2(n-1)}{n(n+1)}$$

$$令 a_n = \frac{A(n)}{n+1}$$

$$\forall n > 0, \qquad a_n = a_{n-1} + \frac{2(n-1)}{n(n+1)}$$

$$Solve\ using\ induction, a_n \approx 2\ln(n)$$

$$A(n) \approx (n+1)2\ln n = (n+1)\ 2\ (\ln 2)\ (\lg n)$$

$$\approx 1.38(n+1)\lg n \in \Theta(n \lg n)$$

矩陣乘法

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \times \begin{bmatrix} j & k & l \\ m & n & o \\ p & q & r \end{bmatrix} =$$

$$\begin{bmatrix} aj + bm + cp & ak + bn + cq & al + bo + cr \\ dj + em + fp & dk + en + fq & dl + eo + fr \\ gj + hm + ip & gk + hn + iq & gl + ho + ir \end{bmatrix}$$

矩陣乘法

問題：求得兩個 $n \times n$ 矩陣的乘積。

輸入：正整數 n，二維數字陣列 A 與 B，這兩個矩陣的行索引及列索引都是由 1 到 n。

輸出：二維數字陣列 C，這個矩陣的行索引及列索引都是由 1 到 n，並含有 A 與 B 的乘積。

```
void matrixmult (int n,
                 const number A[][] ,
                 const number B[][] ,
                 number C[][])
{
    index i, j, k;

    for (i=1; i<=n; i++)
        for (j=1; j<=n; j++){
            C[i][j] = 0;
            for (k=1; k<=n; k++)
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
}
```

$\Theta(n^3)$

Strassen 矩陣乘法

假設我們想得到 A 與 B 兩個 2×2 的矩陣的乘積 C，也就是說，

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

Strassen 證明假設我們令

$$\begin{aligned}
m_1 &= (a_{11} + a_{22})(b_{11} + b_{22}) \\
m_2 &= (a_{21} + a_{22})\, b_{11} \\
m_3 &= a_{11}(b_{12} - b_{22}) \\
m_4 &= a_{22}(b_{21} - b_{11}) \\
m_5 &= (a_{11} + a_{12})\, b_{22} \\
m_6 &= (a_{21} - a_{11})(b_{11} + b_{12}) \\
m_7 &= (a_{12} - a_{22})(b_{21} + b_{22})
\end{aligned}$$

則乘積 C 可由下列式子求得

$$C = \begin{bmatrix} m_1 + m_4 - m_5 + m_7 & m_3 + m_5 \\ m_2 + m_4 & m_1 + m_3 - m_2 + m_6 \end{bmatrix}$$

比較

8 個乘法，4個加減法

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \times \begin{pmatrix} E & F \\ G & H \end{pmatrix} = \begin{pmatrix} AE+BG & AF+BH \\ CE+DG & CF+DH \end{pmatrix}$$

Strassen's Method

$P_1 = A\cdot(F-H)$

$P_2 = (A+B)\cdot H$

$P_3 = (C+D)\cdot E$

$P_4 = D\cdot(G-E)$

$P_5 = (A+D)\cdot(E+H)$

$P_6 = (B-D)\cdot(G+H)$

$P_7 = (A-C)\cdot(E+F)$

$AE+BG = P_5 + P_4 - P_2 + P_6$

$AF+BH = P_1 + P_2$

$CE+DG = P_3 + P_4$

$CF+DH = P_5 + P_1 - P_3 - P_7$

7 個乘法，18個加減法

在大矩陣上用 Strassen 法 (cont.)

$$\begin{array}{c} \leftarrow n/2 \rightarrow \\ n/2 \updownarrow \end{array} \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

圖 2.4　在 Strassen演算法中將大矩陣分割成子矩陣的動作。

$$A_{11} = \begin{bmatrix} a_{11} & a_{12} \cdots a_{1,n/2} \\ a_{21} & a_{22} \cdots a_{2,n/2} \\ & \vdots \\ a_{n/2,1} & \cdots a_{n/2,n/2} \end{bmatrix}$$

在大矩陣上用 Strassen 法 (cont.)

- 利用Strassen的方法，首先我們計算

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

- 以同樣的方式，我們可以算出 M_1 到 M_7
- 藉由M_1到 M_7進一步算出 C_{11}、C_{12}、C_{21} & C_{22}

如：$C_{11} = M_1 + M_4 - M_5 + M_7$

- A 與 B 的乘積 = 合併四個子矩陣得到

範例

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \end{bmatrix} \quad B = \begin{bmatrix} 8 & 9 & 1 & 2 \\ 3 & 4 & 5 & 6 \\ 7 & 8 & 9 & 1 \\ 2 & 3 & 4 & 5 \end{bmatrix}$$

$$\begin{aligned} M_1 &= (A_{11} + A_{22}) \times (B_{11} + B_{22}) \\ &= \left(\begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} + \begin{bmatrix} 2 & 3 \\ 6 & 7 \end{bmatrix} \right) \times \left(\begin{bmatrix} 8 & 9 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 9 & 1 \\ 4 & 5 \end{bmatrix} \right) \\ &= \begin{bmatrix} 3 & 5 \\ 11 & 13 \end{bmatrix} \times \begin{bmatrix} 17 & 10 \\ 7 & 9 \end{bmatrix} \end{aligned}$$

$$\begin{array}{c} \leftarrow 2 \rightarrow \\ \updownarrow 2 \end{array} \left[\begin{array}{c:c} C_{11} & C_{12} \\ \hdashline C_{21} & C_{22} \end{array} \right] = \left[\begin{array}{cc:cc} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ \hdashline 9 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \end{array} \right] \times \left[\begin{array}{cc:cc} 8 & 9 & 1 & 2 \\ 3 & 4 & 5 & 6 \\ \hdashline 7 & 8 & 9 & 1 \\ 2 & 3 & 4 & 5 \end{array} \right]$$

當矩陣已經足夠小了，我們就用標準的方法來相乘。在本例中，我們是在 $n=2$ 時開始使用標準相乘法。於是，

$$
\begin{aligned}
M_1 &= \begin{bmatrix} 3 & 5 \\ 11 & 13 \end{bmatrix} \times \begin{bmatrix} 17 & 10 \\ 7 & 9 \end{bmatrix} \\
&= \begin{bmatrix} 3 \times 17 + 5 \times 7 & 3 \times 10 + 5 \times 9 \\ 11 \times 17 + 13 \times 7 & 11 \times 10 + 13 \times 9 \end{bmatrix} = \begin{bmatrix} 86 & 75 \\ 278 & 227 \end{bmatrix}
\end{aligned}
$$

在此之後，我們以同樣的方法計算 M_2 到 M_7，接著 C_{11}、C_{12}、C_{21} 與 C_{22} 的值都會計算出來。將它們合起來可以得到 C。

Strassen's 矩陣乘法演算法

問題：當 n 為 2 的乘冪時，求出兩個 $n \times n$ 矩陣的乘積。

輸入：一個為 2 的乘冪的整數 n，以及兩個 $n \times n$ 矩陣 A 與 B。

輸出：A 與 B 的乘積 C。

```
void strassen (int n
                   n × n_matrix A,
                   n × n matrix B,
                   n × n_matrix& C)
{
  if (n <= threshold)
     使用標準演算法計算 C = A × B;
  else {
     將 A 切成 4 個子矩陣 A11, A12, A21,A22;
     將 B 切成 4 個子矩陣 B11, B12, B21,B22;
     使用 Strassen 的方式計算 C = A × B;
       // 遞迴呼叫範例 ;
       // strassen (n/2, A11 + A22, B11 + B22, M1);
  }
}
```

Strassen 法的 T(n)

基本運算：一個基本的乘法。

輸入大小：n，也就是這些矩陣的列數與欄數。

為簡化起見，在分析這個情況時，我們會一直將矩陣分割下去，直到得到兩個 1×1 矩陣為止；到此，我們只需要把兩個矩陣裡面的數字相乘。實際上使用的門檻值並不會影響量級。當 n=1 時，只需要做一次的乘法。當輸入是兩個 $n \times n$ 的矩陣時 (n>1)，這個演算法會被呼叫 7 次，每次都會傳遞一個 $(n/2) \times (n/2)$ 的矩陣，並且在最上層時並不會用到乘法。故可得下列的遞迴式

$$T(n) = 7T\left(\frac{n}{2}\right) \quad \text{當 } n>1 \text{ 且 } n \text{ 為 } 2 \text{ 的乘冪}$$
$$T(1) = 1$$

在附錄 B 的範例 B.2 解出這個遞迴式。答案如下

$$T(n) = n^{\lg 7} \approx n^{2.81} \in \Theta\left(n^{2.81}\right)$$

Strassen 法的 T(n)

Table 2.3: A comparison of two algorithms that multiply **n** × **n** matrices

	Standard Algorithm	Strassen's Algorithm
Multiplications	n^3	$n^{2.81}$
Additions/Subtractions	$n^3 - n^2$	$6n^{2.81} - 6n^2$

Strassen 法無論在乘法或加減法的時間複雜度都較原有演算法好

大整數的計算

- 當需要對大小超過電腦硬體表達整數能力的整數執行運算
- 不能使用浮點數！

大整數的計算

- 例如，整數543,127可以表示為下面的陣列S：

5	4	3	1	2	7
$S[6]$	$S[5]$	$S[4]$	$S[3]$	$S[2]$	$S[1]$

- 想要具有表達正負整數的能力，我們只要保留在高位的陣列單元給正負號即可

- 利用此表示法，大整數的加法與減法都是線性時間
- 而簡單的乘法則是平方時間複雜度

大整數的乘法

- 根據divide-and-conquer將一個n位數分割成兩個約為$n/2$位數的整數

$$\underbrace{567,832}_{6\ \text{位數}} = \underbrace{567}_{3\ \text{位數}} \times 10^3 + \underbrace{832}_{3\ \text{位數}}$$

$$\underbrace{9,423,723}_{7\ \text{位數}} = \underbrace{9423}_{4\ \text{位數}} \times 10^3 + \underbrace{723}_{3\ \text{位數}}$$

大整數的乘法 (cont.)

- 如果n為整數u的位數，我們將把該整數分成兩個整數，一個是 位數，一個是 位數，如下所示：

$$\underbrace{u}_{n\ \text{位數}} = \underbrace{x}_{\lceil n/2 \rceil\ \text{位數}} \times 10^m + \underbrace{y}_{\lfloor n/2 \rfloor\ \text{位數}}$$

- 指數m可由下式求得

$$m = \left\lfloor \frac{n}{2} \right\rfloor$$

大整數的乘法 (cont.)

- 假定有兩個n位數整數如下

$$u = x \times 10^m + y$$
$$v = w \times 10^m + z$$

- 它們的乘積則為

$$uv = (x \times 10^m + y)(w \times 10^m + z)$$
$$= xw \times 10^{2m} + (xz + wy) \times 10^m + yz$$

- 在位數約為原來一半的整數上做4次相乘運算以及一些線性時間的運算，就可以得到u與v相乘的結果

範例

- 考慮下面的式子：

$567{,}832 \times 9{,}423{,}723$

$= (567 \times 10^3 + 832)(9{,}423 \times 10^3 + 723)$

$= 567 \times 9423 \times 10^6 + (567 \times 723 + 9423 \times 832) \times 10^3 + 832 \times 723$

- 較小整數的乘積可藉由將其切成更小整數而得到
- 持續分割直到直到到達門檻值，然後再用標準的乘法計算方法

大整數乘法

問題：將兩個大整數 u 與 v 相乘。

輸入：大整數 u 與 v。

輸出：u 與 v 的乘積 $prod$。

```
large_integer prod (large_integer u, large_integer v)
{
  large_integer x, y, w, z;
  int n, m;

    n= maximum(u的位數 ,v的位數 )
    if (u == 0 || v == 0)
       return 0;
    else if (n <= threshold)
       return 由一般方法算出之 u × v的值 ;

    else {
       m = ⌊n/2⌋;
       x = u divide 10^m; y = u rem 10^m;
       w = v divide 10^m; z = v rem 10^m;
       return  prod(x, w)×10^2m + (prod(x, z) + prod(w, y))×10^m+prod(y, z)
  }
}
```

大整數乘法的 Worst Case Time Complexity

基本運算：當相加、相減、或執行 `divide` 10^m、`rem` 10^m、或 `×` 10^m 時，一位數字 (以十進位表示) 的操作。每次對後三者函式的呼叫導致基本運算執行 m 次。

輸入大小：也就是兩個大整數 u、v 的位數。

由於遞迴只有過了 $threshold$ 才會停止，故最差的情況則發生在兩個整數都沒有出現 0 的位數時。我們將分析這個情況。

假設 n 為 2 的乘冪，那麼 x、y、w 與 z 的位數恰好均為 $n/2$，這意味著對 $prod$ 的四個遞迴呼叫的輸入大小都是 $n/2$。因為 $m=n/2$，所以加法、減法、`divide` 10^m、`rem` 10^m、或 `×` 10^m 等線性時間的運算都具有以 n 為單位的線性時間複雜度。對這些線性時間運算來說，它們最大輸入大小並不相同，因此無法直接求得精確的時間複雜度。更簡單的方法，是將所有的線性時間運算組成一項 cn，其中 c 為正的常數。於是我們的遞迴式成為

$$W(n) = 4W\left(\frac{n}{2}\right) + cn \qquad \text{當 } n > s \text{ 且 } n \text{ 為 2 的乘冪}$$
$$W(s) = 0$$

大整數乘法的 Worst Case Time Complexity

- 因為我們討論的是所有輸入的大小均為2的乘冪的情況，故s的實際值，就是當我們停止分割實體時那個n值。它的小於或等於$threshold$，並為2的乘冪。

$$W(n) \in \Theta(n^{\lg 4}) = \Theta(n^2)$$

- 當n不限於2的乘冪的情況，建立的式子仍與前面的遞迴式類似，但式子內會包含下限整數(floor)與上限整數(ceiling)

比平方時間更快？

```
else {
    m = [n/2];
    x = u divide 10^m; y = u rem 10^m;
    w = v divide 10^m; z = v rem 10^m;
    r = prod2(x + y, w + z);
    p = prod2(x, w);
    q = prod2(y, z);
    return p × 10^2m + (r-p-q) × 10^m+q;
}
```

改良後的大整數乘法的 Worst Case Time Complexity

n	x	y	$x+y$	Number of digits for $x+y$
4	10	10	20	$2 = n/2$
4	99	99	198	$3 = (n/2)+1$
8	1,000	1,000	2,000	$4 = n/2$
8	9,999	9,999	19,998	$5 = (n/2)+1$

改良後的大整數乘法的 Worst Case Time Complexity

	Input Size
$prod2(x+y, w+z)$	$\frac{n}{2} \leq \text{input size} \leq \frac{n}{2}+1$
$prod2(x, w)$	$\frac{n}{2}$
$prod2(y, z)$	$\frac{n}{2}$

For $n > s$, n 是 2 的乘冪

$$3W\left(\frac{n}{2}\right) + cn \leq W(n) \leq 3W\left(\frac{n}{2}+1\right) + cn$$

$$W(s) = 0$$

$$W(n) \in \Theta(n^{log_2 3}) \approx \Theta(n^{1.58})$$

決定門檻值

- n 的最佳門檻值(optimal threshold value)。
  - n 要分割到多小？
- 利用 Mergesort 與 Exchangesort 來說明

Mergesort 的最佳門檻值

- Mergesort最差情況的時間複雜度

$$W(n) = W(\left\lfloor \frac{n}{2} \right\rfloor) + W(\left\lceil \frac{n}{2} \right\rceil) + n - 1$$

- 假設 Merge 所需的時間為$32n\ \mu s$
- 實際執行時 Mergesort 的最差情況時間複雜度為：

$$W(n) = W(\left\lfloor \frac{n}{2} \right\rfloor) + W(\left\lceil \frac{n}{2} \right\rceil) + 32n\ \mu s$$

- 只討論n為2的乘冪的情況。我們可以得到下列的遞迴式：

  $W(n) = 2W(n/2) + 32n\ \mu s$　當$n > 1$且n為2的乘冪時

  $W(1) = 0\ \mu s$

- $W(n) = 32n\lg n\ \mu s$

Mergesort 的最佳門檻值 (cont.)

- 修改演算法使其會在 (t為某個門檻值)呼叫交換排序

$$W(n) = \begin{cases} \dfrac{n(n-1)}{2}\ \mu s & \text{對於 } n \le t \\ W(\left\lfloor \dfrac{n}{2} \right\rfloor) + W(\left\lceil \dfrac{n}{2} \right\rceil) + 32n\ \mu s & \text{對於 } n > t \end{cases}$$

- 欲求得t的最佳值，我們必須解出下列的式子

$$W(\left\lfloor \frac{t}{2} \right\rfloor) + W(\left\lceil \frac{t}{2} \right\rceil) + 32t = \frac{t(t-1)}{2}$$

- $\lfloor t/2 \rfloor$與$\lceil t/2 \rceil$都小於等於t

$$W\left(\left\lfloor \frac{t}{2} \right\rfloor\right) = \frac{\lfloor t/2 \rfloor(\lfloor t/2 \rfloor - 1)}{2} \text{ 且 } W(\left\lceil \frac{t}{2} \right\rceil) = \frac{\lceil t/2 \rceil(\lceil t/2 \rceil - 1)}{2}$$

Mergesort 的最佳門檻值 (cont.)

$$\frac{\lfloor t/2 \rfloor(\lfloor t/2 \rfloor - 1)}{2} + \frac{\lceil t/2 \rceil(\lceil t/2 \rceil - 1)}{2} + 32t = \frac{t(t-1)}{2}$$

- t 為偶數
  - $\lfloor t/2 \rfloor$與$\lceil t/2 \rceil$都等於$t/2$
  - $t = 128$
- t 為奇數
  - $\lfloor t/2 \rfloor = \frac{t-1}{2}$ $\lceil t/2 \rceil = \frac{t+1}{2}$
  - $t = 128.008$
- 可得最佳門檻值為 128

何時不能使用 Divide-and-Conquer

- 一個大小為 n 的個體被分成兩個或更多大小接近 n 的個體。
- 一個大小為n的個體被分成 n 個大小為 n/c 的個體，其中c為常數。